Cybersecurity ML Project — Complete A-to-Z Report

Oct 17, 2026 · @Mussab

1. Executive summary

The project's documented final model is **Standard Weighted XGBoost** (XGBoost with `"balanced"` sample weights). On deduplicated NSL-KDD it scored 99.62% accuracy, 0.859 macro F1 and 0.886 balanced accuracy under 5-fold CV. It missed 86 attacks, against 140 for the Original (oversampled) XGBoost. It is paired with an Isolation Forest set to a 1% target false-alarm rate. Source: `reports/final_report.md`, numbers from `results/fair_comparison/cv_summary.csv`.

The project is an NSL-KDD network-intrusion classifier (27–28 attack/normal classes, 40–41 features). It was built in ten notebook phases (`notebooks/phase1…phase10`). Then a leak-free re-evaluation in `src/` corrected the earlier numbers. A CICIDS2017 cross-dataset check came last.

What changed along the way (the story you should tell in a viva):

1. **Phases 1–4 (notebooks).** Train and test files were merged, rare classes (< 10 rows) dropped, and the data re-split 80/20 stratified. Categoricals were label-encoded and features standard-scaled. Small classes were oversampled to 1,000 rows. Twelve models were tested and XGBoost was picked (99.75% accuracy, 0.837 macro F1 on the 29,696-row test split).
2. **Phases 5–8.** Explainability (SHAP, LIME, XGBoost importance), a rule-based threat-response simulation, deep evaluation (ROC/PR, calibration) and integrity testing were run on that model.
3. **Phases 9–10.** Class weighting (Phase 9) and targeted rare-class weights (Phase 10) were tried. The Phase 10 write-ups found **no universal winner** on the old test split.
4. **Leakage fix (`src/`).** 5,086 test rows (17.1%) were exact copies of training rows. The old split was therefore abandoned. Data was deduplicated after merging and the constant `num_outbound_cmds` dropped. All three XGBoost recipes were re-compared with 5-fold CV and on the official NSL-KDD split (which contains attack types unseen in training).
5. **Final decision.** Standard Weighted XGBoost was chosen because it misses fewer attacks and has higher balanced accuracy. The cost is lower accuracy and slightly lower macro F1; that macro-F1 gap is within fold-to-fold noise. The Isolation Forest was added because the classifiers catch only 24–29% of new attack types.

How to read this report. Every approach is tagged with one status:

Tag	Meaning
Implemented	Code exists and was run; outputs exist in the repo
Tested	Evaluated and compared, but not chosen
Rejected	Evaluated and explicitly dropped, with a recorded reason
Selected	The final choice in `reports/final_report.md`
Unverified	Claimed or referenced, but the supporting file or output is missing

Three inconsistencies you must know before a review (details in section 11):

- `models/final_xgboost_model.pkl` is byte-identical (same MD5) to `models/phase9_weighted_xgboost_candidate.pkl`. So the file called "final" is the Phase 9 class-weighted model on the **old, leaky 41-feature scaled pipeline**, not a model trained on cleaned data.
- `src/models/train_final_model.py` and `models/final_xgboost_cleaned_v2_metadata.json` describe the **Original** (oversampled) XGBoost. The final report selects **Standard Weighted** XGBoost. The `.pkl` files that script would write (`final_xgboost_cleaned_v2*.pkl`) are not in `models/`.
- The original `frontend/app.py` hard-coded "Original XGBoost", 99.73% and 0.867. The original `backend/main.py` returned a dummy prediction. Both were rebuilt in this work (section 10).

2. Project structure and file-by-file guide

The repo has two generations of code. Ten exploratory notebooks (`notebooks/`) read and write `data/processed/` , `models/` , `results/phaseN/` and `reports/phaseN/` . A later set of leak-free scripts (`src/`) rebuilds the data from `data/raw/` and writes `results/fair_comparison/` and `results/cicids2017/` . The web app (`backend/` , `frontend/`) serves a model trained by `src/models/train_deployment_model.py` , added in this work.

```
testing-cybersecurity/
├─ data/raw/            kdd_train.csv, kdd_test.csv (+ cicids2017_download/,
not in git)
├─ data/processed/      notebook-pipeline artefacts (encoders, scaler,
npy/csv splits)
├─ notebooks/           phase1 … phase10 .ipynb
├─ src/
|  ├─ preprocessing/    build_dataset.py, cicids2017.py
|  ├─ models/           anomaly.py, train_final_model.py,
train_deployment_model.py*
|  ├─ evaluation/       compare_models.py, hybrid_eval.py, cicids2017_eval.py
|  └─ explainability/, response/, severity/   empty packages (__init__.py
only)
├─ models/              saved XGBoost .pkl files + metadata json (+
deployment/*)
├─ results/             phase4 … phase10, fair_comparison, cicids2017
├─ reports/             phaseN reports, final_report.md, model_review_report
(1).pdf
├─ backend/             FastAPI + MongoDB API*
├─ frontend/            Streamlit app*
└─ requirements.txt, .env.example, cybersecurity.txt, structure.txt
   (* = written or rebuilt in this work, see section 10)
```

2.1 Leak-free source code (`src/`) — produces every final number

File	Purpose	Key functions / sections	Inputs	Outputs	Depends on
`src/preprocessing/build_dataset.py`	Leak-free NSL-KDD preparation	`load_raw()`, `clean_combined()` (merge, drop constant column, dedup, drop < 10-row classes), `encode()` (train-only LabelEncoder, unseen → −1), `build()` (80/20 stratified)	`data/raw/kdd_*.csv`	In-memory DataFrames	pandas, sklearn
`src/preprocessing/cicids2017.py`	CICIDS2017 cleaning	`_load_raw()`, `clean()`; `FAMILIES` dict of 8 attack families	`data/raw/cicids2017_download/MachineLearningCVE/*.csv`	`data/processed/cicids2017_clean.parquet`	pandas, numpy
`src/evaluation/compare_models.py`	**Core experiment.** Fair comparison of 3 XGBoost recipes	`XGB_PARAMS`, `oversample()`, `fit_model()` (the 3 balancing recipes), `cross_validate()`, `official_split()`, `main()`	`build_dataset`	`results/fair_comparison/cv_summary.csv`, `cv_folds.csv`, `cv_per_class.csv`, `official_split_binary.csv`, `official_split_info.json`	build_dataset

File	Purpose	Key functions / sections	Inputs	Outputs	Depends on
`src/models/anomaly.py`	Isolation Forest detector	`IF_PARAMS`, `TARGET_FALSE_ALARM_RATES = (0.01, 0.02, 0.05)`, `_transform()` (log1p), `AnomalyDetector.fit/score/threshold/flag`	normal rows only	fitted object	sklearn
`src/evaluation/hybrid_eval.py`	Classifier OR Isolation Forest	`binary_row()`, `score_all()`, `calibration_rank()`, `cross_validate()`, `official_split()`	compare_models, anomaly	`results/fair_comparison/hybrid_cv.csv`, `hybrid_official_split.csv`	compare_models, anomaly, build_dataset
`src/evaluation/cicids2017_eval.py`	Same 3 recipes + IF on CICIDS2017	`TARGETED_MULTIPLIERS` (Heartbleed 5, Infiltration 3, SQLi 2, XSS 2), `cross_validate()`, `new_attack_test()` (leave one family out)	cicids2017	`results/cicids2017/cv_summary.csv`, `cv_folds.csv`, `cv_per_class.csv`, `hybrid_cv.csv`, `new_attack_families.csv`	compare_models, hybrid_eval, anomaly
`src/models/train_final_model.py`	Trains **Original** XGBoost on all cleaned data	Top-level script; calls `fit_model("Original XGBoost")`	build_dataset	`models/final_xgboost_cleaned_v2*.pkl` (**missing**) + `models/final_xgboost_cleaned_v2_metadata.json` (present)	compare_models
`src/models/train_deployment_model.py` (new)	Trains the **selected** Standard Weighted XGBoost + IF for the app	See section 10	build_dataset	`models/deployment/*`	compare_models, anomaly

2.2 Notebooks (`notebooks/`) — exploratory history

Notebook	What it implements	Main inputs	Main outputs
`phase1_data_understanding.ipynb`	Shapes, column types, class counts, test-only classes, missing values and duplicates, feature ranges	`data/raw/*.csv`	Printed only
`phase2_data_preprocessing.ipynb`	Merge, rare-class filter, 80/20 split, LabelEncoder, StandardScaler	raw CSVs	`data/processed/X_{train,test}_encoded.csv`, `y_*_encoded.npy`, `scaler.pkl`, `feature_encoders.pkl`, `target_encoder.pkl`, `encoding_mapping.json`
`phase3_class_imbalance.ipynb`	Class weights, RandomOverSampler to 1,000, SMOTE check	processed files	`X_train_resampled.npy`, `y_train_resampled.npy`, `X_test_scaled.npy`, `processed_dataset.zip`
`phase4_model_training.ipynb`	11 models (6 classical, 4 boosting, 1 DNN); XGBoost saved	resampled data	`models/final_xgboost_model.pkl` (original version, now `…_original_backup.pkl`), `results/phase4/*`, `reports/phase4/*`
`phase5_explainable_ai.ipynb`	Feature importance, SHAP, LIME, error analysis	Phase 4 model	`results/phase5/*`, `reports/phase5/phase5_explainable_ai_report.txt`

Notebook	What it implements	Main inputs	Main outputs
`phase6_threat_response.ipynb`	Rule-based `SEVERITY_MAPPING` (Low / Medium / High / Critical) → response (Monitor / Alert / Investigate / Immediate Response); unmapped classes default to Medium	Phase 4 model	`results/phase6/phase6_threat_response_system.pkl`, `reports/phase6/phase6_response_report.txt` (10 simulated events)
`phase7_deep_model_evaluation.ipynb`	ROC-AUC and PR per class, calibration, confidence, support-vs-performance correlation	Phase 4 model	`results/phase7/*` (30+ files), `reports/phase7/*`
`phase8_model_testing.ipynb`	Data and model integrity, preprocessing consistency, rare-class and confidence testing, simulated ALLOW / ALERT / REVIEW / INVESTIGATE	Phase 4 model	`results/phase8/*`, `reports/phase8/*`
`phase9_model_improvement.ipynb`	Leakage investigation, rare-class errors, baseline vs class-weighted, selection, threshold policy	`X_train_encoded.csv` (unscaled)	`models/phase9_{baseline,weighted,selected}_xgboost_candidate.pkl`, `results/phase9/*`, `reports/phase9/phase9_final_report.md`
`phase10_final_model_testing.ipynb`	Test-set comparison, audit, rare diagnosis, oversampled and targeted candidates, best-per-attack	Phase 9 models	`models/phase10_*_candidate.pkl`, `results/phase10/*` (50+ files), `reports/phase10/*`

2.3 Models (`models/`)

File	What it actually is (verified by MD5 and `n_features_in_`)
`final_xgboost_model.pkl`	Identical to `phase9_weighted_xgboost_candidate.pkl` (MD5 `d8066414…`): class-weighted XGBoost, 41 **unscaled** features, 28 classes, trained on 95,026 rows of the leaky split. Overwritten 2026-10-15 12:42 outside any notebook
`final_xgboost_model_original_backup.pkl`	Phase 4 Original XGBoost (41 **scaled** features, 28 classes)
`phase9_baseline_xgboost_candidate.pkl`	Phase 9 unweighted baseline
`phase9_selected_xgboost_candidate.pkl`	Same bytes as the weighted candidate
`phase10_oversampled_xgboost_candidate.pkl`	Weighted + oversample-to-100 (step 10.8)
`phase10_targeted_weight_xgboost_candidate.pkl`	Targeted multipliers (step 10.9)
`final_xgboost_cleaned_v2_metadata.json`	Metadata for an Original XGBoost on cleaned data (40 features, 27 classes); its `.pkl` files are missing

File	What it actually is (verified by MD5 and `n_features_in_`)
`deployment/` (new)	Standard Weighted XGBoost + IF for the web app (section 10)

2.4 Reports, results and other files

- `reports/final_report.md` — **the authoritative decision document** (2026-10-15).
- `reports/model_review_report (1).pdf` — an earlier 6-page plain-English review (2026-10-14). It recommended **Original** XGBoost, and that recommendation was reversed by `final_report.md`.
- `reports/phase10.zip` — zip of 4 of the `reports/phase10/` files (duplicate).
- `results/all_results_files_inventory.csv` — file inventory of `results/`. `results/phase5_artifact_verification.csv` and `results/phase5_backup/` are duplicates.
- `cybersecurity.txt` — environment-setup notes (venv, GPU and PyTorch CUDA troubleshooting). Its paths refer to `D:\smart-cybersecurity-ml`, the project's earlier location.
- `structure.txt` — 2.8 MB Windows `tree` dump, mostly of `.venv`.
- `requirements.txt` — unpinned packages. It does not list torch, lightgbm, shap, lime, seaborn, matplotlib or pyarrow, which the notebooks and `cicids2017.py` need.

3. Original dataset and its problems

The main dataset is **NSL-KDD**: `data/raw/kdd_train.csv` (125,973 rows) and `data/raw/kdd_test.csv` (22,544 rows). Each has 42 columns: 41 connection features and the target `labels`. Inspection is in `notebooks/phase1_data_understanding.ipynb`. The download source and licence of the two CSVs are not recorded in the repo.

Features. 3 categorical (`protocol_type`: 3 values, `service`: 70, `flag`: 11) and 38 numeric. The numeric ones are counts, byte totals, flags and 0–1 rates (Phase 1, step 1.2).

Problems found (with the file where each is shown):

#	Problem	Evidence	Where
1	Train has 23 classes, test has 37 total. 14 attack types appear only in test (apache2, httptunnel, mailbomb, mscan, named, processtable, ps, saint, sendmail, snmpgetattack, snmpguess, xlock, xsnoop, xterm).	Phase 1 step 1.3 output	`phase1_data_understanding.ipynb`
2	Extreme class imbalance: normal 67,343 and neptune 41,214 in train, vs spy 2, perl 3, phf 4.	Phase 1 step 1.3	same
3	Very rare classes: combined, 9 classes have < 10 rows (ps 9, sendmail 8, phf 5, perl 4, xlock 4, xterm 3, spy 2, named 2, xsnoop 1).	Phase 2 step 2.3	`phase2_data_preprocessing.ipynb`
4	Heavy-tailed numeric ranges: `src_bytes` up to 1.38e9, `duration` up to 42,908, `count` up to 511.	Phase 1 step 1.5	`phase1_data_understanding.ipynb`
5	Constant column `num_outbound_cmds` (always 0).	Phase 1 ranges; dropped in `src/preprocessing/build_dataset.py`	`CONSTANT_COLS`
6	Duplicates: 0 in train, 31 in test, but **15,791 exact duplicate rows once train and test are combined**.	Phase 1 step 1.4; the 15,791 figure was recomputed for this report from `load_raw()`	`build_dataset.py`
7	**Train–test leakage in the notebook pipeline.** After merging and re-splitting without dedup, 5,086 of 29,696 test rows (17.1%) were copies of training rows. 8 overlapping rows had conflicting labels.	Phase 9 steps 9.2 / 9.2.1	`results/phase9/phase9_step9_2_leakage_investigation.txt`, `phase9_step9_2_1_overlap_label_check.txt`, `phase9_step9_2_2_conflicting_rows.csv`
8	No missing values in either file.	Phase 1 step 1.4	—

Final cleaned NSL-KDD table (from `clean_combined()`, recomputed for this report): 148,517 merged rows. Deduplication leaves 132,726, and dropping classes with < 10 rows leaves **132,679 rows, 27 classes, 40 features**. That is 70,310 normal and 62,369 attack rows.

`ftp_write` survives the notebook pipeline (10 rows) but has only 9 unique rows, so the cleaned pipeline drops it. That is why notebook results show **28** classes and `src/` results show **27**.

Second dataset — CICIDS2017 (implemented, external check only). `data/raw/cicids2017_download/MachineLearningCVE/*.csv` holds 8 daily files and 2,830,743 flows. Per `src/preprocessing/cicids2017.py`, they came from the Hugging Face mirror `bencorn/CICIDS2017` with SHA-256 `c3f26274…`, because the official UNB link redirects to a registration page. Its problems: mis-encoded dashes in web-attack labels, a duplicated column `Fwd Header Length.1`, NaN/infinite values in `Flow Bytes/s` and `Flow Packets/s`, constant columns, duplicates, and imbalance. These raw files and the cleaned parquet are **not tracked in git** (see `.gitignore`).

4. Cleaning, preprocessing, encoding, feature selection and splitting

The project has **two preprocessing pipelines**. The notebook pipeline (Phases 2–3) produced every Phase 4–10 result and was later found to leak. The leak-free pipeline (`src/preprocessing/build_dataset.py`) produced every number in the final report.

Step	Notebook pipeline — *Implemented, later replaced*	Leak-free pipeline — *Selected*
Merge train + test	Yes (step 2.2), to fix the "23 vs 37 classes" problem	Yes (`clean_combined()`)
Remove duplicates	Train only, before merging (step 2.1); removed 0 rows	**After merging**, so no record sits on both sides of a split
Constant column	Kept `num_outbound_cmds` → 41 features	Dropped (`CONSTANT_COLS`) → **40 features**
Rare classes	Dropped classes < 10 rows → 28 classes, 148,479 rows	Same threshold (`MIN_SAMPLES = 10`) → **27 classes, 132,679 rows**
Split	One 80/20 stratified split, `random_state=42` → 118,783 train / 29,696 test	5-fold `StratifiedKFold(shuffle=True, random_state=42)` in `compare_models.py`. `build()` also offers an 80/20 split. Plus the official train/test files for new-attack tests
Categorical encoding	`LabelEncoder` per column, fit on train only (step 2.5); saved to `data/processed/feature_encoders.pkl` and `encoding_mapping.json`	`encode()`: `LabelEncoder` fit on the training fold only; categories unseen in training map to **−1**
Target encoding	`LabelEncoder` → `target_encoder.pkl` (28 classes)	`np.searchsorted(sorted class names)` → integer ids 0–26

Step	Notebook pipeline — *Implemented, later replaced*	Leak-free pipeline — *Selected*
Scaling	`StandardScaler` fit on train (step 2.7) → `scaler.pkl`; used by every Phase 4 model	**None.** XGBoost does not need it (docstring of `build_dataset.py`)
Saved artefacts	`X_train_encoded.csv`, `X_test_encoded.csv`, `y_*_encoded.npy`, `X_test_scaled.npy`, `X_train_resampled.npy`, `processed_dataset.zip`	Nothing saved; data is rebuilt in memory on each run

Feature selection. No automatic feature-selection method (filter, wrapper, RFE, PCA, etc.) is implemented anywhere in the repo. The only reductions are dropping the constant `num_outbound_cmds` (leak-free pipeline) and, for CICIDS2017, dropping constant and duplicated columns. Feature *importance* was analysed (Phase 5, Phase 10 step 10.15) but never used to remove features.

For the Isolation Forest only (`src/models/anomaly.py`, `_transform`): numeric columns get `log1p(clip(x, 0))` so heavy-tailed byte counts don't dominate. On NSL-KDD the three categorical columns are skipped.

CICIDS2017 cleaning (`src/preprocessing/cicids2017.py`, `clean()`): strip column names and fix the web-attack dash; drop `Fwd Header Length.1`; coerce to float32; drop NaN/±inf rows and duplicates per file and again after merging; drop constant columns and classes < 10 rows. It keeps **all 425,726 attack flows plus a random 500,000 BENIGN sample** (seed 42), giving 69 features and 14 classes. The result is cached to `data/processed/cicids2017_clean.parquet`. All features are numeric, so no encoding is needed.

How the final dataset is prepared, end to end:

1. `load_raw()` reads both NSL-KDD CSVs.
2. `clean_combined()` merges them, drops `num_outbound_cmds`, removes exact duplicates and drops classes with fewer than 10 rows.
3. `compare_models.cross_validate()` builds sorted class ids and 5 stratified folds. Inside each fold, `encode()` fits label encoders on the training part only.
4. `fit_model()` applies the model-specific balancing (section 5) to the training part only, then fits XGBoost.
5. Each row is predicted once, by the fold model that never saw it. Metrics are computed over all out-of-fold predictions.

5. Class-imbalance handling

The selected technique is **balanced sample weights** (`compute_sample_weight("balanced", y)`), applied in `fit_model()` in `src/evaluation/compare_models.py`. The training split's imbalance ratio was 7,858.75 (normal 62,870 vs ftp_write 8; `results/phase9/phase9_step9_4_class_weights.csv`). Balanced weights range from 0.067 for normal to 530.3 for ftp_write.

Technique	Where	What it does	Status
Drop classes with < 10 rows	Phase 2 step 2.3; `MIN_SAMPLES` in `build_dataset.py`	Removes classes too small to stratify	**Implemented** (both pipelines)
Balanced class weights (analysis)	Phase 3 step 3.1 (`compute_class_weight`)	Prints weights; not applied to XGBoost in Phase 4	Implemented (analysis only)
Controlled random oversampling to 1,000 rows	Phase 3 step 3.2 (`imblearn RandomOverSampler`); `oversample()` in `compare_models.py`	Duplicates rows of any class below 1,000. 118,783 → 135,441 training rows	**Tested** — this is the "Original XGBoost" recipe; not selected
SMOTE	Phase 3 step 3.3	Only a check: minimum class count after oversampling was 1,000, so the notebook printed "SMOTE is not necessary"	**Rejected** (never run)
`class_weight="balanced"` in sklearn models	Phase 4 step 4.1.2 (LR, DT, RF, SVM)	Applied on top of the oversampled data	Tested
Class-weighted cross-entropy loss	Phase 4 step 4.3 (PyTorch DNN)	`nn.CrossEntropyLoss(weight=class_weights)`	Tested
Balanced sample weights for XGBoost	Phase 9 step 9.5; `fit_model()`	Weights each row by inverse class frequency	**Selected** ("Standard Weighted XGBoost")
Balanced weights + oversampling to 100 rows	Phase 10 step 10.8 → `models/phase10_oversampled_xgboost_candidate.pkl`	Weighted XGBoost on data where classes < 100 are oversampled to 100	Tested; validation macro F1 0.8933 vs 0.8869, not carried forward
Targeted rare-class multipliers	Phase 10 step 10.9; `TARGETED_MULTIPLIERS`	Balanced weights × rootkit 5, multihop 3, ftp_write 2, loadmodule 2	Tested, then **Rejected** in the final report
Confidence-threshold alert policy	Phase 9 step 9.9	Thresholds 0.50–0.99 trade false positives against review cases	Tested (simulation only)
Isolation Forest safety net	`src/models/anomaly.py`	Not imbalance handling as such; it flags anomalies so unseen or rare attacks still get an alert	**Selected** as an add-on at 1% target FAR

Evidence that weighting helps minority classes (Phase 9 validation, 23,757 rows, `results/phase9/phase9_step9_5_validation_comparison.csv`). Balanced accuracy rose from 0.794 to 0.906 and macro F1 from 0.827 to 0.887. Accuracy fell from 0.9974 to 0.9964. Per-class recall rose for land (0 → 1.00), snmpgetattack (0.14 → 0.86), httptunnel (0.71 → 1.00), multihop (0.33 → 0.67) and saint (0.67 → 0.80). rootkit stayed at 0 (`phase9_step9_5_classwise_comparison.csv`).

Caveat. These Phase 9/10 figures come from the old 41-feature pipeline, so they share the leakage problem. The decisive imbalance comparison is the leak-free 5-fold CV in section 6.

6. Models and variants tested, with results

Eleven model families were screened in Phase 4. Three XGBoost variants then went through the leak-free comparison. Every Phase 4–10 number below comes from the **old leaky split** (29,696 test rows, 28 classes, 41 features), so treat it as a screening result, not a final one.

6.1 Phase 4 screening — `notebooks/phase4_model_training.ipynb`

All models were trained on `X_train_resampled.npy` (135,441 oversampled, scaled rows) and tested on `X_test_scaled.npy`. Outputs are printed in the notebook. Only the XGBoost results were saved to files (`results/phase4/final_xgboost_summary.csv`, `final_xgboost_per_class_performance.csv`, `reports/phase4/final_xgboost_classification_report.txt`).

Model	Settings (from code)	Accuracy	Macro F1	Weighted F1	Train time (s)	Status
Gradient Boosting (sklearn)	200 trees, depth 3, lr 0.1	0.9970	**0.8599**	0.9970	2,025.8	Tested; rejected (slow)
Random Forest	200 trees, balanced	0.9960	0.8442	0.9962	4.0	Tested
XGBoost	200 trees, depth 6, lr 0.1, subsample 0.8, colsample 0.8	**0.9975**	0.8370	**0.9974**	36.1	**Selected in Phase 4** → `models/final_xgboost_model.pkl`
Decision Tree	balanced	0.9963	0.8074	0.9963	1.5	Tested
KNN	k = 5	0.9909	0.7767	0.9914	3.0	Tested

Model	Settings (from code)	Accuracy	Macro F1	Weighted F1	Train time (s)	Status
PyTorch DNN	256-128-64, BatchNorm, Dropout 0.3, weighted CE, Adam, early stopping	0.9527	0.6693	0.9636	108.6	Tested; the `dnn_best_model.pth` it saves is not in `models/`
SVM (RBF)	balanced	0.8870	0.5238	0.9203	531.5	Tested
Logistic Regression	balanced, 1,000 iter	0.8726	0.5061	0.9080	81.7	Tested
LightGBM	200 trees, 31 leaves	0.5204	0.1075	0.5989	9.9	Tested; anomalous result never investigated
AdaBoost	200 est., lr 0.1	0.8433	0.0658	0.7772	29.4	Tested
Naive Bayes	Gaussian	0.4230	0.3311	0.4458	0.4	Tested

The notebook has no markdown that states *why* XGBoost was picked over Gradient Boosting, which had the higher macro F1. The evidence in the outputs is the highest accuracy and weighted F1, at about 1/56 of Gradient Boosting's training time. Treat that reasoning as inferred, not recorded. Phase 4 XGBoost got 0 F1 on ftp_write, multihop and rootkit, and 0.59 on saint (`final_xgboost_per_class_performance.csv`).

6.2 Phase 9 — baseline vs class-weighted XGBoost (validation)

Training used 80% of the old training split (95,026 rows, unscaled 41 features) and validation the other 20% (23,757 rows). Source: `results/phase9/phase9_step9_5_validation_comparison.csv` .

Model	Accuracy	Balanced acc.	Macro precision	Macro F1
Baseline XGBoost	0.9974	0.7938	0.8945	0.8267
Class-Weighted XGBoost	0.9964	**0.9057**	**0.9001**	**0.8869**

Selected by "highest validation macro F1, balanced accuracy as tie-breaker" (`phase9_step9_6_model_selection.json`), saved as `models/phase9_selected_xgboost_candidate.pkl` .

6.3 Phase 10 — three candidates on the old test split

Sources: `reports/phase10/phase10_step10_5_final_model_decision.md`, `results/phase10/phase10_step10_12_test_metrics_comparison.csv`, `phase10_step10_17_overall_model_comparison.csv`.

Model	Accuracy	Balanced acc.	Macro F1	Errors
Original XGBoost (Phase 4)	**0.9975**	0.8408	0.8370	**73**
Standard Weighted XGBoost (Phase 9)	0.9964	0.8652	0.8346	107
Targeted Weighted XGBoost (Phase 10.9)	0.9965	**0.8673**	**0.8382**	104

Phase 10's own conclusion was **"No universal winner"** (step 10.5). Step 10.11 nonetheless marked the targeted model "selected for further evaluation; not production-approved". rootkit and multihop stayed at 0 recall for every model on this test set.

6.4 Leak-free comparison — the evidence for the final decision

`python -m src.evaluation.compare_models`: 5-fold stratified CV on the 132,679 deduplicated rows. The same `XGB_PARAMS` are used for all three. The script was run twice, and the two runs disagree slightly:

Metric	Original — run 1	Std Weighted — run 1	Targeted — run 1	Original — run 2	Std Weighted — run 2	Targeted — run 2
Accuracy	**99.73%**	99.62%	99.61%	**99.75%**	99.59%	99.60%
Macro F1	**0.867**	0.859	0.858	**0.875**	0.857	0.853
Macro F1 fold std	0.013	0.015	0.014	0.023	0.015	0.015
Balanced accuracy	0.859	0.886	**0.886**	0.866	0.881	**0.882**
Missed attacks	140	86	**82**	131	91	**80**
False alarms	**112**	309	324	**109**	334	336
Wrong predictions	**355**	504	515	**333**	538	530

- **Run 1** is what `reports/final_report.md` quotes. Its classifier-only rows are still in `results/fair_comparison/hybrid_cv.csv` and in git at commit `d826bba~1`.

- **Run 2** is what is on disk now in `cv_summary.csv`, `cv_folds.csv`, `cv_per_class.csv` and `official_split_binary.csv`. It was written 2026-10-17 22:40 in commit `d826bba` ("update"). The cause of the drift is not recorded. The code is unchanged apart from a `multipliers` parameter, so the likely cause is a different environment or library version, but that is **unverified**.
- Both runs agree on the direction: weighting cuts missed attacks by about a third and roughly triples false alarms. It also raises balanced accuracy and lowers macro F1 slightly.

Minority classes (run 1, `cv_per_class.csv` at `d826bba~1`, recall Original → Standard Weighted): land 0.72 → 1.00, snmpgetattack 0.61 → 0.84, multihop 0.20 → 0.33, imap 0.73 → 0.82, httptunnel 0.93 → 0.95, saint 0.65 → 0.68. Recall fell for loadmodule (0.50 → 0.40) and buffer_overflow (0.88 → 0.85). rootkit stayed at 0.15. Several F1 scores *drop* despite higher recall (snmpgetattack 0.51 → 0.40, buffer_overflow 0.88 → 0.74), because weighting pulls in false positives.

6.5 New attack types — official NSL-KDD split

Trained on `kdd_train.csv`, tested on `kdd_test.csv`: 11,299 attacks, 1,146 of them from 17 types never seen in training, plus 11,245 normal rows (`official_split_info.json`). Run 1 (final report): known types 93.4% for all three models. New types: Original 23.6%, Standard 28.0%, Targeted 29.4%. FAR 0.76% / 0.75% / 0.77%. Run 2 (current `official_split_binary.csv`): new types 22.9% / 27.7% / 30.5%.

6.6 Hybrid classifier + Isolation Forest — `src/evaluation/hybrid_eval.py`

Official split, Standard Weighted (`hybrid_official_split.csv`):

Rule	Known types caught	New types caught	FAR	Missed	False alarms
Classifier only	93.4%	28.0%	0.75%	1,494	84
+ IF @ 1% target	93.5%	**55.5%**	1.49%	1,172	168
+ IF @ 2% target	93.5%	62.7%	2.34%	1,087	263
+ IF @ 5% target	93.9%	69.6%	4.46%	963	501
IF alone @ 1%	74.8%	44.9%	0.92%	3,195	103

On known types under CV (`hybrid_cv.csv`), IF @ 1% cuts misses only from 86 to 65 and raises false alarms from 309 to 961.

6.7 CICIDS2017 — `src/evaluation/cicids2017_eval.py`

All three recipes were retrained on 925,726 flows, 69 features and 14 classes (`results/cicids2017/cv_summary.csv`):

Metric	Original	Standard Weighted	Targeted
Accuracy	**99.84%**	99.81%	99.81%
Macro F1	0.901	**0.914**	0.912
Balanced accuracy	0.900	0.920	**0.923**
Missed attacks	175	46	**45**
False alarms	**506**	725	717

The weakest classes for Standard Weighted are Web Attack XSS (recall 0.51), SQL Injection (0.62) and Brute Force (0.74). On held-out attack families (`new_attack_families.csv`), pooled detection of unseen flows is Original 20.9%, Standard 9.2%, Targeted 9.2%, and Standard + IF @ 1% 10.6%. The Isolation Forest lifts Heartbleed (11 flows) from 0% to 90.9% and Infiltration (36 flows) from 0% to 44.4%.

7. Final model selection — the evidence-based reasoning

Selected: Standard Weighted XGBoost + Isolation Forest at 1% target FAR. The deciding criterion is fewer missed attacks and better minority-class recall at an acceptable false-alarm cost, not accuracy. This is the decision written in `reports/final_report.md`, "Main Model" and "Recommendations".

Why accuracy alone cannot decide. Normal is 53% and neptune 32% of the cleaned data, so a model can score more than 99.5% while missing whole minority classes. In Phase 4, XGBoost had 99.75% accuracy yet 0 F1 on three classes. The project therefore tracks macro F1 (every class counts equally), balanced accuracy (mean per-class recall), missed attacks and false alarms.

Decision table (leak-free 5-fold CV, run 1 / run 2):

Criterion	Winner	Evidence
Missed attacks (security cost)	Weighted models	Original 140 / 131; Standard 86 / 91; Targeted 82 / 80
Balanced accuracy (minority recall)	Weighted models	0.859 / 0.866 vs 0.886 / 0.881

Criterion	Winner	Evidence
New attack types, official split	Weighted models	23.6% vs 28.0% (Standard), 29.4% (Targeted)
False alarms (analyst workload)	Original	112 / 109 vs 309 / 334
Accuracy	Original	99.73% / 99.75% vs 99.62% / 99.59%
Macro F1	Original	0.867 / 0.875 vs 0.859 / 0.857; the run-1 gap of 0.008 is below the fold std of 0.013–0.015
CICIDS2017 known attacks	Standard Weighted	Macro F1 0.914 vs 0.901; missed 46 vs 175
CICIDS2017 unseen families	Original	Pooled 20.9% vs 9.2%, mainly from unseen DDoS (62.9% vs 16.9%); the ranking varies by family

Reasoning chain, as recorded:

1. A missed attack is treated as costlier than a false alarm. That is a stated design priority ("Missing an attack is more dangerous than a false alarm"), not a measured cost.
2. On that priority, both weighted models beat the Original: about 39% fewer misses and higher balanced accuracy.
3. Standard vs Targeted: Targeted saves only 4 misses in run 1 (82 vs 86), costs 15 more false alarms, and has lower macro F1. Its multipliers are hand-tuned to rare classes (and ftp_write no longer exists after deduplication). So the simpler Standard model is kept. **Targeted is rejected.**
4. The classifiers catch only about 1 in 4 new-type attacks. The Isolation Forest at 1% target doubles that, 28% → 55.5%, for +0.74 points of FAR (84 → 168 false alarms). The 1% setting is chosen because the 2% and 5% settings add false alarms faster than catches.

What changed from earlier phases, and why:

Phase	Claimed winner	Why it was superseded
4	Original XGBoost (99.75%)	Evaluated on a test set with 17.1% copies of training rows; minority classes at 0 F1
9	Class-Weighted XGBoost (validation macro F1 0.887)	Same leaky data

Phase	Claimed winner	Why it was superseded
10.5	"No universal winner"	Same leaky test set, reused after Phase 8
10.11	Targeted candidate	Small gains on 2–3-row classes; not confirmed by leak-free CV
Final	**Standard Weighted + IF @ 1%**	Leak-free CV, official new-attack split, CICIDS2017 known-attack check

Honest limits of the decision:

- Under run 2, the Original's macro-F1 lead (0.875 vs 0.857) is about one fold std (0.015–0.023). The "not a clear difference" statement is weaker there than in run 1.
- The CICIDS2017 leave-one-family-out test favours neither model. The final report says so explicitly.
- No trained Standard Weighted model on the cleaned 40-feature data existed in `models/` before this work. The decision was made from CV results. Section 10 adds `src/models/train_deployment_model.py` to produce one.

8. Explainable AI (XAI)

XAI was run once, in Phase 5 (`notebooks/phase5_explainable_ai.ipynb`). It explains the **Phase 4 Original XGBoost on the old scaled 41-feature test split**, not the final Standard Weighted model. The summary report is `reports/phase5/phase5_explainable_ai_report.txt` , and the outputs are in `results/phase5/` (a duplicate copy sits in `results/phase5_backup/`). `src/explainability/` is an empty package: no XAI code exists outside the notebook.

Method	How it was run	Output file	Top findings
XGBoost built-in importance (gain, weight, cover, total_gain)	`get_booster().get_score()` ; f0…f40 mapped back to names	`phase5_feature_importance_{gain,weight,cover,total_gain,named}.csv` , `phase5_feature_importance_top20.png`	By gain: land, urgent, same_srv_rate, wrong_fragment, num_failed_logins. By weight (split count) and total gain: src_bytes first
SHAP	`shap.TreeExplainer` on a random sample of **200** test rows; mean |SHAP| across rows and classes	`phase5_shap_global_importance.csv` / `.png`	src_bytes 0.832, service 0.561, dst_host_srv_count 0.416, dst_bytes 0.398, dst_host_count 0.343, protocol_type 0.325
LIME	`LimeTabularExplainer` , one test record (sample 0, normal, confidence 0.99999)	`phase5_lime_individual_explanation{,_named}.csv` / `.png`	src_bytes ⩽ 0, hot ⩽ 0, logged_in > 1, dst_host_same_srv_rate > 1, num_root ⩽ 0

Method	How it was run	Output file	Top findings
Error analysis	Confusion matrix and misclassification patterns	`phase5_confusion_matrix.csv`, `phase5_misclassification_patterns.csv`, `phase5_misclassified_records.csv`, `phase5_errors_by_actual_class.csv`	73 errors in 36 patterns. saint ↔ satan is the largest (7 each way); normal → satan 4, normal → snmpgetattack 4, nmap → ipsweep 4

Phase 10 step 10.15 also saved normalised importance for a Phase 10 model (`results/phase10/phase10_step10_15_top20_feature_importance.csv`; top: land 0.144, urgent 0.103, wrong_fragment 0.095). Which candidate it used is not stated in the saved CSV.

How to explain it in a viva:

- Gain importance and SHAP disagree. Gain rewards features used in a few highly informative splits (land, urgent: rare but decisive for specific attacks). SHAP measures average contribution over real samples, where byte volumes and service dominate. Both are valid but answer different questions.
- The LIME conditions are in **standardised units** (the model saw `StandardScaler` output). So "logged_in > 1.00" means "more than 1 standard deviation above the mean", not a raw value. The `_named` CSV only renames columns; it does not convert units.

Not done (state this as a limitation): no XAI on the final Standard Weighted model or on cleaned data. No SHAP per class. No SHAP stability check across samples (only 200 rows). No explanation for the Isolation Forest.

9. Execution workflow and result traceability

The final results can be regenerated with four commands from the project root. The notebooks are history: they depend on `data/processed/` files produced by earlier notebooks, so they must be run in phase order.

A. Reproduce the final evidence (leak-free):

1. `python -m src.evaluation.compare_models`: CV and official-split comparison, about 3 × 5 + 3 XGBoost fits.
2. `python -m src.evaluation.hybrid_eval`: adds the Isolation Forest.
3. `python -m src.preprocessing.cicids2017`, then `python -m src.evaluation.cicids2017_eval`. Needs the CICIDS2017 CSVs, which are not in git.
4. `python -m src.models.train_deployment_model`: writes the app's model bundle (took 2 min 35 s on this machine).

B. Notebook history: run `phase1` → `phase10` in order, from inside `notebooks/` (they use `../data`).

C. Web app: see section 10.

Trace any number back to its code and file:

Result	Value(s)	Output file	Produced by
Raw shapes, test-only classes	125,973 / 22,544 rows; 14 test-only types	printed	`phase1_data_understanding.ipynb` steps 1.1, 1.3
Cleaned dataset size	132,679 rows, 27 classes, 40 features	in memory	`build_dataset.clean_combined()`
Leakage	5,086 test rows (17.1%) copied from train	`results/phase9/phase9_step9_2_leakage_investigation.txt`	`phase9` step 9.2
Oversampling	118,783 → 135,441 rows	`data/processed/X_train_resampled.npy`	`phase3` step 3.2
Phase 4 model table	XGBoost 99.75%, macro F1 0.837	printed; `results/phase4/final_xgboost_summary.csv`	`phase4` steps 4.1.3, 4.2.2
SHAP / LIME	src_bytes 0.832 …	`results/phase5/phase5_shap_global_importance.csv`, `…lime…csv`	`phase5`
Severity / response simulation	10 events	`reports/phase6/phase6_response_report.txt`	`phase6`
ROC / PR / calibration	log loss 0.0099, top-3 acc 0.9997	`results/phase7/*.json/csv`	`phase7`
Integrity tests	29,623 correct / 73 wrong	`results/phase8/*`	`phase8`
Baseline vs weighted (validation)	macro F1 0.827 → 0.887	`results/phase9/phase9_step9_5_validation_comparison.csv`	`phase9` step 9.5
"No universal winner"	99.75% vs 99.64%	`reports/phase10/phase10_step10_5_final_model_decision.md`	`phase10` step 10.5
Final CV table	run 1: 140 / 86 / 82 missed	git `d826bba~1:results/fair_comparison/cv_summary.csv` ; `hybrid_cv.csv`	`compare_models.cross_validate()`, `hybrid_eval.cross_validate()`
Final CV table (current file)	run 2: 131 / 91 / 80 missed	`results/fair_comparison/cv_summary.csv`	`compare_models.cross_validate()`
New-attack detection	23.6 / 28.0 / 29.4%	`hybrid_official_split.csv` (classifier-only rows)	`compare_models.official_split()`, `hybrid_eval.official_split()`
IF @ 1%	new types 28.0% → 55.5%	`results/fair_comparison/hybrid_official_split.csv`	`hybrid_eval.official_split()`

Result	Value(s)	Output file	Produced by
CICIDS2017	macro F1 0.914, 46 missed	`results/cicids2017/cv_summary.csv`	`cicids2017_eval.cross_validate()`
Unseen families	pooled 9.2%	`results/cicids2017/new_attack_families.csv`	`cicids2017_eval.new_attack_test()`
Deployed model	132,679 rows, IF threshold 0.5997	`models/deployment/standard_weighted_xgboost_metadata.json`	`src/models/train_deployment_model.py`

Dependency chain: `build_dataset` → `compare_models` (`fit_model`, `XGB_PARAMS`) → `hybrid_eval` (+ `anomaly`) → `cicids2017_eval` (+ `cicids2017`). `train_deployment_model` reuses `build_dataset`, `compare_models.fit_model` and `anomaly`, so the deployed model is built with exactly the recipe that was evaluated.

10. Web application — Streamlit + FastAPI + MongoDB

The app serves the **selected model** (Standard Weighted XGBoost + Isolation Forest @ 1%), trained on the cleaned data by the evaluated recipe. It was built and tested in this work. Before it, `backend/main.py` returned a hard-coded dummy prediction and `frontend/app.py` was static HTML with fixed numbers.

Architecture: Streamlit (`frontend/app.py`, port 8501) → HTTP/JSON → FastAPI (`backend/main.py`, port 8000) → `ModelService` (loads `models/deployment/standard_weighted_xgboost_bundle.pkl`) and MongoDB (`smart_cybersecurity.predictions`).

File	Role
`src/models/train_deployment_model.py`	Trains the classifier with `fit_model("Standard Weighted XGBoost")` on all 132,679 rows. Fits the `AnomalyDetector` on the 70,310 normal rows. Saves one bundle (model, encoders, feature and class names, detector, metadata) plus a metadata JSON
`backend/config.py`	Settings from env or `.env` (`MONGODB_URI`, `MONGODB_DATABASE`, `MODEL_BUNDLE_PATH`, `MAX_BATCH_ROWS`)
`backend/model_service.py`	Same encoding as `build_dataset.encode()`: unseen category → −1, returned as a warning. Returns class, confidence, top 5 probabilities, IF score vs threshold, and the hybrid alert. Severity and response use the Phase 6 policy

File	Role
`backend/schemas.py`	Pydantic schema for the 40 features: counts ≥ 0, rates in [0, 1], binary flags 0/1
`backend/database.py`	Lazy MongoDB client. If MongoDB is down, predictions still work and `stored` is false
`backend/main.py`	Endpoints, listed below
`frontend/app.py`	Pages: Predict, Batch CSV, History, Model performance, About
`tests/test_api.py`	7 API tests (MongoDB stubbed): all pass in about 15 s

API endpoints: `GET /health` · `GET /model/info` · `GET /model/performance` (reads `results/fair_comparison/*.csv` as saved) · `POST /predict?store=true` · `POST /predict/batch` (up to 5,000 records) · `GET /examples?label=&n=` (real rows from `kdd_test.csv` with true labels) · `GET /predictions?limit=` · `GET /predictions/stats`. Interactive docs are at `http://127.0.0.1:8000/docs`.

Decision logic per record:

1. If the classifier predicts an attack class, the verdict is **Attack** and severity comes from the Phase 6 map. Unmapped classes default to Medium, as in the notebook.
2. Otherwise, if the IF score is above the threshold (0.5997, the 99th percentile of held-out normal scores), the verdict is **Anomaly**, severity Medium, response Alert. This rule for the anomaly-only case is **new in this work**; Phase 6 had no anomaly input.
3. Otherwise the verdict is **Normal**, severity Low, response Monitor.

How to run (from the project root, Windows PowerShell):

```
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m src.models.train_deployment_model      # once, ~3 min
uvicorn backend.main:app --host 127.0.0.1 --port 8000
streamlit run frontend/app.py                    # second terminal
python -m pytest tests -q                        # optional
```

MongoDB must be running at `mongodb://127.0.0.1:27017`; on this machine it is the Windows `MongoDB` service. Copy `.env.example` to `.env` to change settings.

Verified on 2026-10-17:

- `/health` reported the model loaded and the database connected.

- A real `guess_passwd` test row came back as guess_passwd (confidence 0.99993), High / Investigate, and was stored and listed by `/predictions`.
- All 5 Streamlit pages rendered with no exceptions (`streamlit.testing.AppTest`), and loading a smurf example then pressing Predict worked.
- The full `kdd_test.csv` (22,544 rows) scored in 5.4 s through `/predict/batch`: 99.83% of attacks alerted, 2.01% of normal rows alerted. **These rates are optimistic and are not a result**, because those rows are part of the deployment model's training data. Cite section 6 for performance.

Limitations of the app: it scores offline records only (no packet capture or flow extraction), blocks nothing, and has no authentication.

11. Missing or unverifiable information, limitations, and viva preparation

Missing or unverifiable:

Item	Status
Why `cv_summary.csv` changed between run 1 (final report) and run 2 (commit `d826bba`)	Not recorded. Code is effectively unchanged; the environment or library version is suspected but unverified
Who overwrote `models/final_xgboost_model.pkl` with the Phase 9 weighted model, and why (2026-10-15 12:42)	Not done in any notebook or script in the repo
`models/final_xgboost_cleaned_v2*.pkl` (from `train_final_model.py`)	Metadata exists; the model files do not
`models/dnn_best_model.pth` (Phase 4 DNN)	Not in `models/`
Why Phase 4 chose XGBoost over Gradient Boosting (higher macro F1)	Not written down; inferred from accuracy and training time
The LightGBM result (macro F1 0.108)	Anomalous and never investigated
Source and licence of the NSL-KDD CSVs	Not recorded
Library versions used for the reported runs	`requirements.txt` is unpinned. The current venv has Python 3.14.3, xgboost 3.4.1, scikit-learn 1.9.1, pandas 3.0.6

Item	Status
Conflicting rows	Phase 9 reports 8 overlapping rows with conflicting labels; the PDF review says 30 identical-looking rows. Different definitions; not reconciled

Limitations to state openly:

- NSL-KDD is an old, synthetic benchmark. Results do not prove real-network performance.
- Rare classes have 10–15 rows (loadmodule, multihop, rootkit, imap), so their recall is unstable; rootkit recall is only 0.15.
- New-attack detection is weak: 28% for the classifier alone, 55.5% with the IF, and only 9–11% for unseen CICIDS2017 families.
- No hyperparameter tuning was done; all XGBoost variants share one fixed setting.
- The XAI covers the old Phase 4 model only.

Likely viva questions and evidence-based answers:

1. *Why not just report 99.75% accuracy?* That figure came from a test set where 17.1% of rows were copies of training rows. The leak-free CV gives 99.62%, and accuracy hides minority classes anyway.
2. *Why Standard Weighted rather than the Original, which has higher accuracy and macro F1?* It misses about 39% fewer attacks (86 vs 140) and has higher balanced accuracy (0.886 vs 0.859). The macro-F1 gap (0.008) is below the fold std. The cost is about 200 extra false alarms over 70,310 normal rows.
3. *Why not Targeted?* It saves 4 misses for 15 more false alarms, has lower macro F1, and its multipliers were hand-tuned.
4. *Why oversampling to 1,000 and not SMOTE?* After random oversampling the smallest class had 1,000 rows, so SMOTE was judged unnecessary. It was never run, so no SMOTE comparison exists.
5. *What does the Isolation Forest add?* It doubles new-type detection on the official split (28% → 55.5%) for +0.74 points of FAR. It barely helps on known attacks or high-volume unseen families.
6. *Is there data leakage now?* Duplicates are removed after merging, and encoders are fit inside each training fold. The official split additionally tests attack types unseen in training.
7. *Which features matter?* By SHAP: src_bytes, service, dst_host_srv_count, dst_bytes. By gain: land, urgent, wrong_fragment, the rare but decisive flags. These come from the Phase 4 model.

8. *Why are your numbers in two places slightly different?* Run 1 vs run 2 of the same script (section 6.4). Both runs give the same ranking on missed attacks and balanced accuracy.